

Backlog — Tiger Kick (รายเฟส)

รวมงานย่อยของทุกเฟสเป็น backlog พร้อม ID — ใช้สร้างการ์ดใน Kanban / Issues ได้ทันที ไฟล์ 05_Backlog.csv และ 05_Backlog.xlsx ใช้ import เข้า GitHub Projects / Trello / Notion

คำเริ่มต้น: ทุกการ์ดเริ่มที่คอลัมน์ **Backlog** — Priority ปรับได้ตามจริง

Phase 0 — Setup & Networking Foundation

Milestone: **M0 Networking** · Priority เริ่มต้น: **High**

- TK-P0-01 ติดตั้ง Godot 4.6 (เลือก build มาตรฐาน หรือ .NET หากใช้ C#)
- TK-P0-02 สร้าง GitHub repo + เพิ่ม .gitignore ของ Godot และตั้งโครงสร้างโฟลเดอร์ (scenes/, scripts/, assets/)
- TK-P0-03 สร้าง MainMenu.tscn พร้อมปุ่ม Host / Join และช่องกรอก IP:Port
- TK-P0-04 เขียน NetworkManager (autoload) สร้าง ENetMultiplayerPeer สำหรับ host และ client
- TK-P0-05 สร้าง TestArena.tscn (พื้น + วงกลม) และสลับ scene หลังเชื่อมต่อสำเร็จ
- TK-P0-06 ทดสอบเปิดสองอินสแตนซ์ให้เชื่อมกันและพิมพ์ log ยืนยันการเชื่อมต่อ

Acceptance (Definition of Done ของเฟส): - clone repo แล้วเปิดใน Godot ได้ทันทีโดยไม่มี error - เปิดสองอินสแตนซ์ กด Host และ Join แล้วเชื่อมต่อกันสำเร็จ - มี log/ข้อความยืนยันว่า peer เข้ามาแล้ว

Phase X — Development Infrastructure (แทรกระหว่าง Phase 0 กับ 1)

Milestone: **MX Dev Infra** · Priority เริ่มต้น: **Medium**

- TK-PX-01 เขียน Logger เป็น Autoload รับระดับ log และเขียนลงไฟล์ที่ user://logs/
- TK-PX-02 ทำ Debug overlay/console เปิด-ปิดด้วยปุ่มลัด (เช่น F3) แสดง FPS/ping/state
- TK-PX-03 ทำ Performance overlay ใช้ค่าจาก Performance singleton ของ Godot
- TK-PX-04 ทำ ConfigManager อ่าน/เขียน ConfigFile (user://settings.cfg)
- TK-PX-05 ทำเมนู Settings: Graphics, Audio, Controls (rebind ปุ่ม)
- TK-PX-06 วางแนวทาง error handling รวมศูนย์ (จับ error สำคัญและ log)
- TK-PX-07 ติดตั้ง GUT (Godot Unit Test) และเขียนเทสต์ตัวอย่างสำหรับ NetworkManager/State

Acceptance (Definition of Done ของเฟส): - เปิด-ปิด Debug overlay ได้ และเห็น FPS/ping/state แบบเรียลไทม์ -

เปลี่ยน Settings แล้วบันทึก/โหลดกลับมาได้หลังปิดเปิดเกม - มี unit test อย่างน้อย 1 ชุดที่รันผ่าน (เช่น การเปลี่ยนสถานะบทบาท)

Phase 1 — Core Movement & Sync

Milestone: **M1 Movement** · Priority เริ่มต้น: **High**

- TK-P1-01 สร้าง Player.tscn (CharacterBody3D + Camera3D + MultiplayerSynchronizer)
- TK-P1-02 เขียน input การเคลื่อนที่ + Sprint (ปรับความเร็วขณะกด Shift)
- TK-P1-03 ตั้งค่า camera rig แบบ third person (spring arm)
- TK-P1-04 ตั้ง MultiplayerSpawner ให้ spawn ผู้เล่นตาม peer id
- TK-P1-05 กำหนด multiplayer authority ให้แต่ละผู้เล่นควบคุมตัวเอง
- TK-P1-06 ปรับ MultiplayerSynchronizer ให้ซิงค์เฉพาะ property ที่จำเป็น (ตำแหน่ง/หัน)

Acceptance (Definition of Done ของเฟส): - ผู้เล่น 4-8 คนวิ่งพร้อมกันได้ และทุกเครื่องเห็นตำแหน่งตรงกัน - Sprint ทำงานและรู้สึกต่างจากเดินปกติ - ไม่มีอาการ teleport/กระตุกรุนแรงในเน็ตเวิร์กปกติ

Phase 2 — Core Loop (สำคัญที่สุด)

Milestone: **M2 Core Gameplay** · Priority เริ่มต้น: **High**

- TK-P2-01 ทำ Kick: input + hitbox ชั่วคราว + cooldown
- TK-P2-02 ทำ Tag detection: Area3D รอบเสา ตรวจสอบผู้เล่นที่เข้าใกล้
- TK-P2-03 ทำ Tag Sequence (state machine ย่อย): 1) เสาจับล็อกผู้เล่น 2) หุ่นลงพื้น 3) เล่นเอฟเฟกต์กลายร่าง 4) ผู้เล่นที่ถูกหุ้มนกลายเป็นเสาใหม่ 5) เสาเดิมกลับเป็นคน 6) เสาเดิมย้ายออกนอก Safe Circle 7) กลับสู่สถานะ Playing
- TK-P2-04 สร้าง Role state machine (Outer / Tiger) และสลับ property/กล้องเมื่อจบ Tag Sequence
- TK-P2-05 สลับกล้อง First Person (เสา) กับ Third Person (ผู้เล่นอื่น) ตามบทบาท
- TK-P2-06 ทำ Safe Circle เป็นขอบเขตและ logic ที่เกี่ยวข้อง
- TK-P2-07 ทำ RoundManager: จับเวลา/เขื่อนไขว้รอบ และรีเซ็ตเริ่มรอบใหม่
- TK-P2-08 ย้ายการตัดสินใจ Tag/Kick ไปที่ host: client ส่งคำสั่งผ่าน RPC แล้ว host ยืนยันผล

Acceptance (Definition of Done ของเฟส): - เล่นวน core loop ได้ครบ: เตะ → Tag Sequence (จับ/หุ้มน/กลายร่าง) → สลับเสา → เริ่มรอบใหม่ - ทุกเครื่องเห็น Tag Sequence และการเปลี่ยนบทบาทตรงกัน ไม่มีเสาซ้อน/หาย - เสาเดิมกลับเป็นคนและย้ายออกนอก Safe Circle อย่างถูกต้องทุกเครื่อง - กล้องสลับถูกต้องตามบทบาทเมื่อจบ Tag Sequence

Phase 3 — Playtest & Tuning

Milestone: **M3 First Playtest** · Priority เริ่มต้น: **Medium**

- TK-P3-01 แยกค่าพารามิเตอร์สำคัญออกเป็น export variable / resource ปรังง่าย
- TK-P3-02 ทำ build สำหรับแจกทดสอบ (export preset)
- TK-P3-03 จัด session ทดสอบและบันทึกผล (แบบสอบถามสั้น + คลิป)
- TK-P3-04 ปรับค่าและทดสอบซ้ำเป็นรอบ ๆ ตามหัวข้อ Open Questions ใน GDD

Acceptance (Definition of Done ของเฟส): - มีข้อสรุปชัดเจนว่า core loop สนุกพอจะไปต่อหรือไม่ - ได้ชุดค่าพารามิเตอร์ที่ผู้เล่นส่วนใหญ่รู้สึกสนุก

Phase 4 — Feel & Polish

Milestone: **M4 Polish** · Priority เริ่มต้น: **Medium**

- TK-P4-01 import โมเดล/แอนิเมชันจาก Mixamo ผ่าน glTF และตั้ง AnimationTree
- TK-P4-02 ผูก event เสียง (เตะ/จับ/สลับบทบาท) กับ AudioStreamPlayer
- TK-P4-03 ทำ particle/VFX ตอนสลับบทบาทให้เห็นชัดว่าใครเป็นเสือ
- TK-P4-04 ทำ HUD: เวลารอบ, ตัวบ่งชี้บทบาท, feedback ตอนถูกจับ

Acceptance (Definition of Done ของเฟส): - การเตะ/จับ/สลับบทบาทมี feedback ภาพ+เสียงที่ชัด - ผู้เล่นเข้าใจสถานะบทบาททันทีจากภาพ/UI - มีคลิปโมเมนต์ที่ “อยากแชร์” เกิดขึ้นจริงระหว่างทดสอบ

Phase 5 — Feature Expansion & Steam

Milestone: **M5 Ship** · Priority เริ่มต้น: **Medium**

- TK-P5-01 ติดตั้ง GodotSteam และตั้งค่า Steam App ID
- TK-P5-02 ทำ Steam Lobby (สร้าง/เข้าร่วม) แทน/เสริมการต่อ IP ตรง
- TK-P5-03 เพิ่ม Steam Achievements ตามโมเมนต์เด่นในเกม
- TK-P5-04 จัดลำดับความสำคัญฟีเจอร์ที่เหลือด้วยเกณฑ์ความสนุก แล้วทำทีละอย่าง

Acceptance (Definition of Done ของเฟส): - ผู้เล่นสร้าง/เข้าร่วม lobby ผ่าน Steam และเล่นด้วยกันได้ - ฟีเจอร์ใหม่แต่ละตัวผ่านเกณฑ์ความสนุกก่อนเข้าเกมจริง

วิธี import

- **GitHub Projects:** เปิด Project → ปุ่ม ... → *Import* → เลือก `05_Backlog.csv` (แนบ Status/Priority เป็น field)
- **Trello:** ใช้ Power-Up หรือ import CSV แปลงเป็นการ์ด (คอลัมน์ Title = ชื่อการ์ด)
- **Notion:** New → Import → CSV → เลือก `05_Backlog.csv`